DSA: base problems and what they unlock

About 95 problems carry all the transfer. Solve those cold and roughly 440 LeetCode problems stop being separate problems — they become variations you can derive at the table.

How to read this

A **base problem** is one where the idea is born. A **dependent** is a problem that reuses that idea with one thing changed — a constraint, a direction, a dimension. If you can state what changed, you can usually write the dependent without solving it from scratch.

The column that matters most is the middle one. It names the invariant — the single sentence the whole family hangs on. If you can say that sentence out loud before you write code, you own the family. If you can only recall the code, you own one problem.

How to study from this:

1. Solve the base cold, no hints, and write the invariant in your own words.
2. For each dependent, spend 10–15 minutes on paper answering only *what changed and what breaks* — don't do a full submit. Code it only if your answer feels shaky.
3. Come back to the base a week later. If the invariant is gone, the dependents were wasted.

The 3:1 ratio is the point. Most people invert it — 300 submissions, 20 invariants retained. Problem numbers below are LeetCode. A few dependents appear under two bases; that overlap is real and worth noticing, not a mistake.

Arrays, hashing, prefix sums, sliding window

Seven bases. This block alone covers most of the array section of any interview list.

Base	The invariant you must own	Dependents it unlocks
1 Two Sum	Store what you've seen; for each element look up its complement. The map answers "has the partner already passed?"	167 Two Sum II, 170 Two Sum III, 653 Two Sum IV (BST), 15 3Sum, 16 3Sum Closest, 18 4Sum, 259 3Sum Smaller, 454 4Sum II
560 Subarray Sum Equals K	sum(i..j) = P[j] − P[i−1], so a subarray condition becomes a lookup on prefix sums seen so far	974 Subarray Sums Divisible by K, 523 Continuous Subarray Sum, 525 Contiguous Array, 930 Binary Subarrays With Sum, 1248 Count Nice Subarrays, 437 Path Sum III, 1074 Submatrices Sum Target, 303 / 304 Range Sum Query
53 Maximum Subarray (Kadane)	The best subarray ending at i is either this element alone or the previous best extended	152 Maximum Product Subarray, 918 Max Sum Circular Subarray, 1749 Max Absolute Sum, 978 Longest Turbulent Subarray, 1191 K-Concatenation Max Sum, 121 Best Time to Buy and Sell Stock
3 Longest Substring Without Repeating	Variable window: expand right always, shrink left only while the window is invalid. Each index enters and leaves once	159 At Most Two Distinct, 340 At Most K Distinct, 904 Fruit Into Baskets, 424 Longest Repeating Char Replacement, 1004 Max Consecutive Ones III, 209 Min Size Subarray Sum, 1456 Max Vowels (fixed window)
76 Minimum Window Substring	Carry a need-map plus a `formed` counter so validity is O(1) to check, not O(k) to recompute	567 Permutation in String, 438 Find All Anagrams, 30 Substring with Concatenation of All Words
992 Subarrays with K Different Integers	Exactly(K) = atMost(K) − atMost(K−1). "Exactly" windows aren't directly shrinkable, so you never write one	930 Binary Subarrays With Sum, 1248 Count Nice Subarrays, 1358 Substrings Containing All Three Characters
49 Group Anagrams	Pick a canonical form (sorted string, count tuple) and let the hash map do the grouping	242 Valid Anagram, 249 Group Shifted Strings, 438 Find All Anagrams, 128 Longest Consecutive Sequence (set-as-index trick)

The one that pays most here is 560. Prefix-sum-plus-map shows up disguised as a tree problem (437), a matrix problem (1074) and a divisibility problem (974), and almost nobody recognises it across those three.

Two pointers, intervals, sweep line

Base	The invariant you must own	Dependents it unlocks
167 Two Sum II	On sorted data, left and right pointers each move in one direction only — moving left can only increase the sum, moving right can only decrease it	15 3Sum (fix one, two-pointer the rest), 18 4Sum (fix two), 16 3Sum Closest, 259 3Sum Smaller, 11 Container With Most Water, 125 Valid Palindrome, 680 Valid Palindrome II, 977 Squares of a Sorted Array, 283 Move Zeroes
75 Sort Colors	Three-way partition: one pass, three regions, and you only advance the middle pointer when you didn't swap from the right	215 Kth Largest (quickselect partition), 905 Sort Array By Parity, 26 / 80 Remove Duplicates I and II, 27 Remove Element
56 Merge Intervals	Sort by start. Then the only question per interval is whether it overlaps the one you're currently holding	57 Insert Interval, 986 Interval List Intersections, 759 Employee Free Time, 252 Meeting Rooms, 228 Summary Ranges
435 Non-overlapping Intervals	Sort by **end**, not start. Greedily keeping the earliest-finishing interval always leaves the most room	452 Minimum Arrows to Burst Balloons, 646 Maximum Length of Pair Chain, 1353 Max Events That Can Be Attended

253 Meeting Rooms II	Separate starts from ends and sweep. Peak concurrency is a running counter, not a geometry problem	1094 Car Pooling, 1109 Corporate Flight Bookings, 729 / 731 / 732 My Calendar I, II, III, 218 The Skyline Problem

The fork to internalise: **sort by start** when you're merging or intersecting, **sort by end** when you're choosing a maximum non-conflicting set, **sweep** when you care about how many are active at once. Three intervals problems, three different sorts, and picking wrong is where most people lose the round.

Matrix and in-place implementation

These get dismissed as "just implementation", which is why people fail them. There are three real ideas here — shrinking boundaries, composing simple transforms, and encoding extra state inside the data — and each one carries a family.

Base	The invariant you must own	Dependents it unlocks
54 Spiral Matrix	Four boundaries that shrink after each pass. The only bug surface is the single-row and single-column check in the middle of the loop	59 Spiral Matrix II, 885 Spiral Matrix III, 2022 Convert 1D Array Into 2D Array
48 Rotate Image	A rotation is a composition of two trivial operations — transpose, then reverse each row. Don't rotate four cells at a time unless asked	867 Transpose Matrix, 1886 Matrix Obtained By Rotation, 189 Rotate Array
73 Set Matrix Zeroes	O(1) space means storing your markers *inside* the data — here, the first row and column — and handling their own overlap last	289 Game of Life (two states packed into one int), 41 First Missing Positive (sign as the marker), 442 Find All Duplicates
189 Rotate Array	Reverse the whole thing, then reverse each part. Three reversals, no extra array	61 Rotate List, 151 Reverse Words in a String, 186 Reverse Words in a String II
31 Next Permutation	From the right, find the first descent, swap it with the smallest larger element to its right, then reverse the suffix	556 Next Greater Element III, 60 Permutation Sequence, 46 Permutations (the lexicographic ordering)
36 Valid Sudoku	One pass, three index derivations — row, column, and box as `(r / 3) * 3 + c / 3`. Deriving the box index is the whole question	37 Sudoku Solver, 51 N-Queens (diagonal index derivation), 1222 Queens That Can Attack the King
240 Search a 2D Matrix II	Start at the top-right: each comparison eliminates an entire row or an entire column. Not binary search — a staircase walk	74 Search a 2D Matrix (genuinely different — one flat sorted array), 378 Kth Smallest in a Sorted Matrix, 1428 Leftmost Column with at Least a One

73 and 289 being the same trick is worth saying out loud: both ask for O(1) extra space, and both answers are "encode the second state inside the first". One does it with a sentinel row, the other with the second bit of an integer.

Binary search

Four bases, and the third one is the highest-leverage single idea on this page.

Base	The invariant you must own	Dependents it unlocks
704 Binary Search (+ lower / upper bound)	Write one template where the loop invariant is "the answer lives in [lo, hi)" and never guess the ±1 again	35 Search Insert Position, 34 First and Last Position, 278 First Bad Version, 744 Smallest Letter Greater Than Target, 540 Single Element in Sorted Array, 74 Search a 2D Matrix
33 Search in Rotated Sorted Array	With one rotation, at least one half is always sorted — identify which, then ask if the target lies inside it	81 Search in Rotated II (duplicates break the O(log n) guarantee), 153 / 154 Find Minimum in Rotated, 162 Find Peak Element, 852 Peak Index in a Mountain Array
875 Koko Eating Bananas	Binary search the **answer**, not the array. Needs a monotone predicate: if speed x works, every speed above x works	1011 Capacity to Ship Packages, 410 Split Array Largest Sum, 1482 Min Days to Make Bouquets, 1552 Magnetic Force Between Balls, 1231 Divide Chocolate, 774 Minimize Max Distance to Gas Station, 1283 Smallest Divisor Given a Threshold
378 Kth Smallest in a Sorted Matrix	Binary search over the **value space**, with `count(<= x)` as the predicate. The answer need not be an element you ever enumerate	668 Kth Smallest in Multiplication Table, 719 Find K-th Smallest Pair Distance, 1201 Ugly Number III, 4 Median of Two Sorted Arrays (partition search — related but genuinely its own idea)

875 is worth real time. The recognition signal is a question phrased as *minimise the maximum* or *maximise the minimum*, or any "what's the smallest X such that it still fits". Once you hear that phrasing, the hard part is only writing `feasible(x)` honestly — the search around it is four lines you already know.

Stacks, monotonic structures, heaps

Base	The invariant you must own	Dependents it unlocks
739 Daily Temperatures	Keep the stack monotonic. Anything you pop has found its answer; anything still inside is still waiting	496 / 503 Next Greater Element I and II, 901 Online Stock Span, 1762 Buildings With an Ocean View, 456 132 Pattern, 316 Remove Duplicate Letters, 402 Remove K Digits

84 Largest Rectangle in Histogram	Every bar's maximal rectangle is bounded by the first smaller bar on each side — one stack finds both boundaries	85 Maximal Rectangle, 42 Trapping Rain Water, 907 Sum of Subarray Minimums, 2104 Sum of Subarray Ranges
239 Sliding Window Maximum	A deque holding indices in decreasing value order: the front is the answer, and anything smaller arriving behind is already dead	1438 Longest Subarray with Abs Diff <= Limit, 862 Shortest Subarray with Sum at Least K, 1696 Jump Game VI
394 Decode String	Nesting means a stack of *contexts* — push the partial state on `(`, pop and combine on `)`	224 / 227 / 772 Basic Calculator I, II, III, 150 Evaluate Reverse Polish Notation, 1106 Parsing a Boolean Expression, 20 Valid Parentheses
215 Kth Largest Element	Size-k heap gives O(n log k); quickselect gives O(n) average. Know when each is the right answer to give	347 Top K Frequent Elements, 973 K Closest Points to Origin, 692 Top K Frequent Words, 703 Kth Largest in a Stream
23 Merge k Sorted Lists	A heap of k frontier elements turns k-way merging into repeated "pop the global minimum"	378 Kth Smallest in a Sorted Matrix, 373 K Pairs with Smallest Sums, 632 Smallest Range Covering K Lists, 21 Merge Two Sorted Lists
295 Find Median from Data Stream	Two heaps facing each other, sizes kept within one. The median is always at the boundary	480 Sliding Window Median, 1825 Finding MK Average
621 Task Scheduler	With a cooldown, the most frequent element dictates the schedule — the answer is a formula on max frequency, or a greedy heap	767 Reorganize String, 358 Rearrange String k Distance Apart, 1405 Longest Happy String

84 and 42 being the same idea is worth stating plainly — two famously hard problems, one "find the nearest smaller element on both sides" stack.

Linked lists and cyclic sort

Base	The invariant you must own	Dependents it unlocks
206 Reverse Linked List	Three pointers — prev, curr, next — and the loop body is always the same four lines	92 Reverse Linked List II, 25 Reverse Nodes in k-Group, 234 Palindrome Linked List, 143 Reorder List, 24 Swap Nodes in Pairs
142 Linked List Cycle II	Floyd: after they meet, a pointer from the head and one from the meeting point advance at the same speed and collide at the cycle entry	141 Linked List Cycle, 287 Find the Duplicate Number (array as a function graph), 202 Happy Number
876 Middle of the Linked List	Fast moves two, slow moves one — slow lands where you need to split, reverse or compare	19 Remove Nth Node From End, 143 Reorder List, 234 Palindrome Linked List, 148 Sort List
21 Merge Two Sorted Lists	A dummy head removes every head-is-null special case. If your linked-list code has branches for the first node, you skipped this lesson	23 Merge k Sorted Lists, 2 Add Two Numbers, 445 Add Two Numbers II, 86 Partition List, 148 Sort List, 328 Odd Even Linked List
448 Find All Numbers Disappeared	When values are 1..n, the array *is* a hash table — put each value at its own index and read the mismatches	442 Find All Duplicates, 41 First Missing Positive, 268 Missing Number, 645 Set Mismatch, 287 Find the Duplicate Number

287 sits in two families deliberately — it's solvable by cyclic sort reasoning *and* by Floyd. Being able to give both, and say which one respects the "don't modify the array" constraint, is exactly the follow-up interviewers use it for.

Trees

Nine bases. Nearly every tree question is one of these with a different return type.

Base	The invariant you must own	Dependents it unlocks
104 Maximum Depth	Post-order composition: ask both children for their answer, combine, return upward. The whole family differs only in *what* you return	110 Balanced Binary Tree, 543 Diameter, 124 Max Path Sum, 687 Longest Univalue Path, 250 Count Univalue Subtrees, 333 Largest BST Subtree
236 LCA of a Binary Tree	Return the node if found, null if not. A node with a non-null answer from both sides *is* the LCA — no parent pointers, no second pass	235 LCA of a BST, 1644 LCA II (nodes may be absent), 1650 LCA III (parent pointers), 1676 LCA IV (k nodes), 1123 LCA of Deepest Leaves, 865 Smallest Subtree with All Deepest Nodes, 2096 Step-by-Step Directions
113 Path Sum II	Root-to-leaf DFS carrying mutable state down, and undoing it on the way out. The backtrack line is the whole lesson	112 Path Sum, 257 Binary Tree Paths, 129 Sum Root to Leaf Numbers, 437 Path Sum III (prefix sums along the path), 988 Smallest String From Leaf, 1448 Count Good Nodes
102 Level Order Traversal	Capture `queue.size()` before the inner loop — that's what turns a flat BFS into levels	103 Zigzag Level Order, 107 Level Order II, 199 Right Side View, 515 Largest Value in Each Row, 116 / 117 Populating Next Right Pointers, 637 Average of Levels, 863 All Nodes Distance K, 1161 Max Level Sum
94 Inorder Traversal, iterative	An explicit stack replaces the call stack. "Now do it without recursion" is a standard follow-up, not a trick — and Morris traversal is the O(1)-space escalation after it	144 / 145 Preorder and Postorder iterative, 173 BST Iterator, 341 Flatten Nested List Iterator, 284 Peeking Iterator, 285 Inorder Successor in BST
98 Validate BST	Push the valid (min, max) range downward. Comparing only parent-to-child is the classic wrong answer	230 Kth Smallest in a BST, 173 BST Iterator, 450 Delete Node in a BST, 701 Insert into a BST, 700 Search in a BST, 108 Sorted Array to BST, 285 Inorder Successor in BST

105 Construct from Preorder and Inorder	The first preorder element is the root; its position in inorder splits left from right. An index map makes it O(n)	106 Construct from Inorder and Postorder, 889 Construct from Preorder and Postorder, 1008 BST from Preorder, 654 Maximum Binary Tree, 297 Serialize and Deserialize
337 House Robber III	Tree DP returning a *pair* — (best if I take this node, best if I don't) — instead of a single number	968 Binary Tree Cameras, 124 Max Path Sum, 979 Distribute Coins in Binary Tree
208 Implement Trie	A node is a map from character to node plus an end-of-word flag. Prefix queries become a walk, not a scan	211 Design Add and Search Words, 212 Word Search II, 648 Replace Words, 677 Map Sum Pairs, 1268 Search Suggestions System, 421 Maximum XOR of Two Numbers (bit trie)

The LCA row is worth special attention. Five numbered problems, one function, and the mutations — node may not exist, you have parent pointers, there are k nodes — are exactly how a good interviewer escalates within a single round.

Graphs

Base	The invariant you must own	Dependents it unlocks
200 Number of Islands	A grid is a graph with implicit edges. Flood fill each unvisited cell and count how many times you had to start	695 Max Area of Island, 1254 Number of Closed Islands, 1020 Number of Enclaves, 130 Surrounded Regions, 733 Flood Fill, 417 Pacific Atlantic Water Flow, 827 Making a Large Island
994 Rotting Oranges	Multi-source BFS: seed the queue with *every* source at once and the level count is the simultaneous answer	542 01 Matrix, 286 Walls and Gates, 1091 Shortest Path in Binary Matrix, 1162 As Far from Land as Possible
127 Word Ladder	The node is a **state**, not a place. BFS over (position + whatever else can change), and `visited` is keyed on the whole state, not the position	752 Open the Lock, 433 Minimum Genetic Mutation, 1293 Shortest Path with Obstacle Elimination, 864 Shortest Path to Get All Keys, 909 Snakes and Ladders, 773 Sliding Puzzle
210 Course Schedule II	Kahn's: repeatedly take a node with in-degree zero. If you can't empty the graph, there's a cycle	207 Course Schedule, 269 Alien Dictionary, 310 Minimum Height Trees, 802 Find Eventual Safe States, 1136 Parallel Courses, 329 Longest Increasing Path in a Matrix
547 Number of Provinces	Union-find with path compression and union by rank. Use it when edges arrive over time or you only need connectivity, not paths	684 Redundant Connection, 721 Accounts Merge, 990 Satisfiability of Equality Equations, 839 Similar String Groups, 947 Most Stones Removed, 1319 Make Network Connected, 305 Number of Islands II, 1202 Smallest String With Swaps
743 Network Delay Time	Dijkstra: pop the smallest tentative distance, and a node popped is final. Breaks on negative edges	1631 Path With Minimum Effort, 778 Swim in Rising Water, 1514 Path with Maximum Probability, 505 The Maze II, 1976 Number of Ways to Arrive at Destination, 787 Cheapest Flights Within K Stops (Bellman-Ford — the k constraint is why Dijkstra fails)
1584 Min Cost to Connect All Points	MST: Kruskal is sort-edges-plus-union-find, Prim is a heap. Use when every node must be connected at minimum total weight	1135 Connecting Cities With Minimum Cost, 1489 Critical and Pseudo-Critical Edges, 1192 Critical Connections (Tarjan bridges)
133 Clone Graph	A visited map from old node to new node makes the recursion terminate and the sharing correct	138 Copy List with Random Pointer, 1485 Clone Binary Tree With Random Pointer
785 Is Graph Bipartite	Two-colour as you traverse; a conflict means an odd cycle	886 Possible Bipartition, 1042 Flower Planting With No Adjacent

The distinction to be fluent in: **BFS** for unweighted shortest path, **Dijkstra** for non-negative weights, **Bellman-Ford** when there's a hop limit or negative edges, **union-find** when you only need "are these connected" and never the path. 787 exists purely to see whether you know the second and third apart.

Backtracking

Five bases, and they differ only in the loop shape and the dedup rule.

Base	The invariant you must own	Dependents it unlocks
78 Subsets	At each index you either include or exclude. Record at every node of the tree, not only at the leaves	90 Subsets II (sort, then skip equal siblings — not equal ancestors), 77 Combinations, 784 Letter Case Permutation
46 Permutations	Order matters, so every unused element is a candidate at every depth. Track used[] or swap in place	47 Permutations II (same sibling-skip rule as 90), 31 Next Permutation (a different, non-recursive idea), 526 Beautiful Arrangement
39 Combination Sum	Pass `i` to allow reuse, `i + 1` to forbid it. That one character is the entire difference between the family members	40 Combination Sum II, 216 Combination Sum III, 377 Combination Sum IV (counting — becomes DP, not backtracking)
131 Palindrome Partitioning	Try every cut point; recurse on the suffix only if the prefix is valid	93 Restore IP Addresses, 140 Word Break II, 132 Palindrome Partitioning II (the DP version)
79 Word Search	Mark the cell, recurse, unmark. The undo must	212 Word Search II (trie-pruned), 51 N-Queens, 37 Sudoku Solver, 698

	happen on *every* return path, including failures	Partition to K Equal Sum Subsets, 17 Letter Combinations, 22 Generate Parentheses

The dedup rule is worth stating precisely because it's the most common interview slip: sort the input, then at a given depth skip `nums[i]` when `i > start && nums[i] == nums[i-1]`. You're skipping a duplicate *sibling*, never a duplicate *ancestor*. Say that sentence in the room and the follow-up is over.

Greedy and two-pass sweeps

The hardest part of greedy isn't the code, it's justifying it. Every row below has an exchange argument behind it: *swapping my choice for any other choice never improves the answer*. If you can't state that sentence, you're guessing, and a good interviewer will find out in one follow-up.

Base	The invariant you must own	Dependents it unlocks
55 Jump Game	Track the farthest index reachable so far. If the loop index ever passes it, you're stuck — no DP needed	45 Jump Game II (same sweep, but count window boundaries), 1306 Jump Game III, 1024 Video Stitching, 1326 Minimum Number of Taps, 871 Minimum Refueling Stops (greedy + heap)
763 Partition Labels	Precompute each character's last occurrence; a partition can close only when the sweep reaches the furthest last-occurrence seen	1288 Remove Covered Intervals, 1353 Max Events That Can Be Attended, 56 Merge Intervals
406 Queue Reconstruction by Height	Sort on one dimension so the second dimension becomes a direct insert position. Choosing the sort key *is* the algorithm	354 Russian Doll Envelopes, 1029 Two City Scheduling, 881 Boats to Save People, 455 Assign Cookies
134 Gas Station	If the total is non-negative a solution exists; and any failure point means every station before it also fails, so restart after it	53 Maximum Subarray, 1013 Partition Array Into Three Parts With Equal Sum
238 Product of Array Except Self	Two sweeps — prefix left to right, suffix right to left — let each position see both sides without division and without nesting	135 Candy, 42 Trapping Rain Water (the two-pass version), 724 Find Pivot Index, 334 Increasing Triplet Subsequence, 1109 Corporate Flight Bookings (difference array)

The interval greedies (435, 452, 253) live in the intervals section, and 621 Task Scheduler in the heaps section — they belong to this family too, just indexed where you'd look for them.

Dynamic programming

Thirteen bases, split across eight shapes. This is where the leverage is highest and where most people instead memorise fifty individual recurrences.

Base	The invariant you must own	Dependents it unlocks
70 Climbing Stairs	The whole of 1D DP: define f(i) over a prefix, write the transition, then collapse the array to two variables	746 Min Cost Climbing Stairs, 91 Decode Ways, 509 Fibonacci, 1137 Tribonacci
198 House Robber	Take-or-skip with an adjacency constraint. Two running values, not an array	213 House Robber II (circular — run it twice, excluding one end each time), 337 House Robber III (tree), 740 Delete and Earn (bucket by value, then it *is* 198)
322 Coin Change	Unbounded knapsack: loop coins outer, capacity ascending inner, so a coin can be reused	518 Coin Change II (same loops, counting — combinations), 377 Combination Sum IV (loops swapped — permutations), 279 Perfect Squares, 139 Word Break, 983 Minimum Cost For Tickets
416 Partition Equal Subset Sum	0/1 knapsack: iterate capacity **downward** so each item is used once. The loop direction is the entire difference from 322	494 Target Sum, 1049 Last Stone Weight II, 474 Ones and Zeroes (two capacities), 698 Partition to K Equal Sum Subsets
300 Longest Increasing Subsequence	O(n²) DP, then the patience version: `tails[k]` = smallest tail of an increasing subsequence of length k+1, maintained by binary search	354 Russian Doll Envelopes (sort with a tie-break trick, then LIS), 646 Maximum Length of Pair Chain, 673 Number of LIS, 368 Largest Divisible Subset, 1626 Best Team With No Conflicts
188 Best Time to Buy and Sell Stock IV	One state machine: hold or free, × transactions used. Every other stock problem is this with a parameter pinned	121 (k = 1), 122 (k = infinity), 123 (k = 2), 309 (cooldown — one extra state), 714 (fee — subtract on sell)
1143 Longest Common Subsequence	Two-string grid: characters match → take the diagonal + 1; don't match → best of the two skips	72 Edit Distance, 583 Delete Operation for Two Strings, 712 Minimum ASCII Delete Sum, 516 Longest Palindromic Subsequence (LCS of s and reversed s), 1312 Minimum Insertion Steps, 115 Distinct Subsequences, 97 Interleaving String
10 Regular Expression Matching	Same grid, but `*` forks into "use zero copies" and "use one more copy"	44 Wildcard Matching
62 Unique Paths	Grid DP: each cell depends on the cell above and the cell to the left	63 Unique Paths II, 64 Minimum Path Sum, 120 Triangle, 931 Minimum Falling Path Sum, 221 Maximal Square, 1277 Count Square Submatrices
5 Longest Palindromic Substring	Expand around each of the 2n−1 centres, or dp[i][j] from dp[i+1][j−1]	647 Palindromic Substrings, 516 Longest Palindromic Subsequence, 132 Palindrome Partitioning II

312 Burst Balloons	Interval DP with the key reframe: iterate over which element you handle **last**, not first, so the two sides stay independent	1039 Minimum Score Triangulation, 1547 Minimum Cost to Cut a Stick, 375 Guess Number Higher or Lower II, 546 Remove Boxes
139 Word Break	DP over split points with a dictionary set — a string version of coin change	140 Word Break II (memoised backtracking), 343 Integer Break
698 Partition to K Equal Sum Subsets	Bitmask DP: the subset of used elements is the state, because n <= ~20	847 Shortest Path Visiting All Nodes, 526 Beautiful Arrangement, 1986 Minimum Sessions to Finish the Tasks

Two pairs are worth drilling until they're reflex, because interviewers use them as the discriminator. **322 vs 416** — the capacity loop direction decides unbounded versus 0/1. **518 vs 377** — the loop *order* decides combinations versus permutations. Candidates who can state both out loud are rare.

Design problems

These are the highest-value hours on this page per minute spent. They're also where the follow-ups go furthest — thread safety, TTL, distribution — which makes them the bridge from DSA into LLD and system design rounds.

Base	The invariant you must own	Dependents it unlocks
146 LRU Cache	Hash map for lookup plus a doubly linked list for order. Every operation is O(1) because you never search the list — the map hands you the node	460 LFU Cache, 432 All O(1) Data Structure, 1472 Design Browser History, 355 Design Twitter
380 Insert Delete GetRandom O(1)	Array for uniform random access plus an index map. Delete by swapping with the last element	381 Insert Delete GetRandom — Duplicates Allowed, 710 Random Pick with Blacklist, 528 Random Pick with Weight
359 Logger Rate Limiter	A rate limit is a window. Know the fixed window, sliding log and token bucket versions and the memory trade between them	362 Design Hit Counter, 933 Number of Recent Calls
155 Min Stack	Carry the auxiliary answer alongside the data, so the invariant survives every push and pop	716 Max Stack, 232 Implement Queue using Stacks, 225 Implement Stack using Queues, 895 Maximum Frequency Stack
297 Serialize and Deserialize Binary Tree	Pre-order with explicit null markers is enough to reconstruct. One traversal without markers is not	449 Serialize and Deserialize BST, 428 Serialize and Deserialize N-ary Tree, 105 Construct from Preorder and Inorder
706 Design HashMap	Buckets plus chaining, and a load factor that triggers a resize. Know what a collision actually costs	705 Design HashSet, 1206 Design Skiplist

The follow-up ladder on 146 is the one to rehearse end to end: make it thread-safe, add per-entry TTL, turn it into LFU, then distribute it and explain what breaks. That one problem can fill a 45-minute round on its own, and it's among the most-asked design questions at senior level.

Bit manipulation and order statistics

Base	The invariant you must own	Dependents it unlocks
136 Single Number	XOR cancels equal pairs and preserves the odd one out	137 Single Number II (count bits mod 3), 260 Single Number III (split by a differing bit), 268 Missing Number, 389 Find the Difference
191 Number of 1 Bits	`n & (n - 1)` clears the lowest set bit — so the loop runs once per set bit, not 32 times	338 Counting Bits, 231 Power of Two, 201 Bitwise AND of Numbers Range, 78 Subsets (bitmask enumeration)
307 Range Sum Query — Mutable	A Fenwick tree or segment tree buys O(log n) updates at the cost of O(log n) queries, when a prefix array would be O(n) per update	315 Count of Smaller Numbers After Self, 493 Reverse Pairs, 327 Count of Range Sum, 699 Falling Squares
912 Sort an Array (merge sort)	Counting during the merge step: when you take from the right half, you've just learned something about every remaining element on the left	315 Count of Smaller Numbers After Self, 493 Reverse Pairs, 327 Count of Range Sum, 148 Sort List, 23 Merge k Sorted Lists

315, 493 and 327 are the same problem three times, and each has two correct solutions — BIT or merge-sort counting. Being able to offer both, and say why coordinate compression is needed for the BIT version, is a strong senior signal.

String algorithms, math and the remaining tools

Base	The invariant you must own	Dependents it unlocks
28 Find the Index of the First Occurrence (KMP)	The prefix function: for each position, the longest proper prefix that is also a suffix. It tells you how far to fall back instead of restarting	459 Repeated Substring Pattern, 214 Shortest Palindrome, 1392 Longest Happy Prefix, 686 Repeated String Match
187 Repeated DNA Sequences (rolling hash)	A window's hash updates in O(1) on slide — drop the leading term, shift, add the new one. Collisions are a real risk, so verify	1044 Longest Duplicate Substring (hash + binary search on length), 718 Maximum Length of Repeated Subarray, 572 Subtree of Another Tree
71 Simplify Path	A stack is the canonical model for anything path-shaped: push a component, pop on `..`, ignore `.`	388 Longest Absolute File Path, 588 Design In-Memory File System, 1166 Design File System

451 Sort Characters By Frequency	When the key range is bounded, bucket by count instead of sorting — O(n) instead of O(n log n)	347 Top K Frequent Elements, 692 Top K Frequent Words, 164 Maximum Gap (pigeonhole buckets), 75 Sort Colors (counting sort)
179 Largest Number	Define the comparator by the result you actually want, not the natural order. Here: `a + b` versus `b + a` as strings	937 Reorder Data in Log Files, 1029 Two City Scheduling, 406 Queue Reconstruction by Height, 881 Boats to Save People
398 Random Pick Index	Reservoir sampling: keep the i-th item with probability 1/i and the distribution comes out uniform without knowing n in advance	382 Linked List Random Node, 384 Shuffle an Array (Fisher-Yates), 528 Random Pick with Weight, 470 Implement Rand10 Using Rand7
50 Pow(x, n)	Halving the exponent turns a linear loop into a logarithmic one. The same halving idea drives division and square roots	372 Super Pow, 29 Divide Two Integers, 69 Sqrt(x), 204 Count Primes (sieve)
8 String to Integer (atoi)	The algorithm is trivial; the test is whether you enumerate the cases *before* writing code. Treat the spec as the problem	65 Valid Number, 68 Text Justification, 273 Integer to English Words, 166 Fraction to Recurring Decimal, 6 Zigzag Conversion

Manacher's algorithm belongs near 5 Longest Palindromic Substring, but it's rarely required — expand-around-centre at $O(n^2)$ is accepted almost everywhere. Learn KMP properly instead; it shows up more often and the prefix function is reusable.

What doesn't transfer, and the coverage math

Some problems are genuinely one-offs. They're still asked, so do them — just don't expect them to unlock anything.

- **Genuine one-offs**: 332 Reconstruct Itinerary (Hierholzer's algorithm, and almost nothing else uses it), 7 Reverse Integer, 13 / 12 Roman numeral conversion, 171 / 168 Excel Sheet Column Number and Title, 326 / 342 Power of Three and Four.
- **Concurrency**: 1114 Print in Order, 1115 Print FooBar Alternately, 1117 Building H2O, 1195 Fizz Buzz Multithreaded. These aren't DSA at all, they're synchronisation primitives. Some LLD loops ask them; most algorithm rounds don't.

Spiral matrix, set matrix zeroes, next permutation, atoi and text justification are often filed under "implementation-heavy" and skipped. They sit in the matrix and string sections above instead, because each of them does in fact anchor a family. "Implementation-heavy" usually means the idea is unfamiliar, not absent.

The arithmetic. 95 bases above, carrying 421 dependent entries — about 365 distinct once you remove the ones listed under two families. Call it 440 distinct problems of coverage, and a real-world hit rate closer to 85% than 100% — the remaining 15% is the list just above, plus genuinely novel Hards.

Where the 95 sit:

Block	Bases	Dependents listed
Arrays, hashing, prefix sums, windows	7	40
Two pointers, intervals, sweep	5	28
Matrix and in-place implementation	7	21
Binary search	4	22
Stacks, monotonic, heaps	8	33
Linked lists, cyclic sort	5	23
Trees	9	55
Graphs	9	44
Backtracking	5	18
Greedy and two-pass sweeps	5	19
Dynamic programming	13	52
Design	6	18
Bits, order statistics	4	17
String algorithms, math, remaining tools	8	31
Total	**95**	**421**

At three bases a week — base solved cold, dependents reasoned through on paper — that's about eight months. At six a week it's four. The failure mode isn't speed, it's doing the dependents as full submissions instead of as *what changed* exercises, which turns 300 problems back into 300 problems.